PASSWORD POLICY VALIDATION ENGINE

Python Programming Project Report

Project topic	Password Policy Validation Engine
Language	Python 3
Purpose	Check sample passwords against configurable validation rules and explain failures

Includes problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, source code, test cases, sample console output, and conclusion.

1. Problem Statement

Develop a Python-based Password Policy Validation Engine that checks a set of sample password strings against a configurable password policy. The engine must evaluate minimum length, the presence of lowercase and uppercase letters, digits and special characters, repeated characters, and prohibited substrings. It must identify every failed condition for each invalid sample and display a final count of valid and invalid passwords. For privacy, the program should not print the entered password back to the screen.

2. Objectives

• Validate password length against a minimum requirement of 8 characters.

• Require at least one lowercase letter, uppercase letter, digit, and special character.

• Reject three or more identical characters occurring consecutively.

• Reject passwords containing configured prohibited substrings, checked without case sensitivity.

• Explain each failed validation rule rather than displaying only a pass/fail result.

• Summarize how many sample passwords were checked, accepted, and rejected.

3. Assumptions

• The minimum length is set to 8 characters and can be changed in the program.

• A special character is any character in Python's string.punctuation set.

• The prohibited substrings are password, admin, and 123456; the check is case-insensitive.

• Three or more identical characters in a row are not allowed.

• The program is an educational validator and does not store passwords or connect to an authentication system.

• Passwords are sample inputs only; real credentials should not be entered into an untrusted demonstration program.

4. Algorithm

1. Start the program and display the policy rules.

2. Read the number of sample passwords; stop with an error if the count is invalid or negative.

3. Initialize valid and invalid counters to zero.

4. For each sample password, create an empty list of validation errors.

5. Check minimum length, lowercase, uppercase, digit, and special-character requirements.

6. Check whether any character repeats three or more times consecutively.

7. Check for prohibited substrings without case sensitivity.

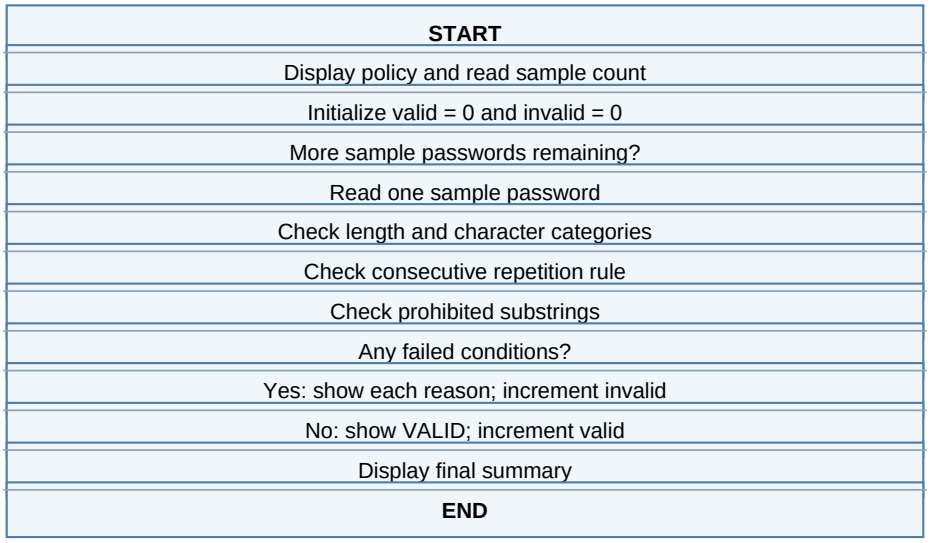
8. If errors exist, mark the password invalid and display each failed condition; otherwise mark it valid.

9. Update the appropriate counter and continue until all samples are checked.

10. Display the total checked, valid, and invalid counts, then stop.

5. Flowchart

The flowchart below shows the sequence of checks. Every validation check contributes an error message when its condition fails; the program completes all checks so it can explain multiple failures.



Flow: Start → Read count → Process each sample → Run all checks → Report all failures or success → Update counters → Repeat → Summary → End.

6. Pseudocode

```
BEGIN
    SET minimum_length = 8
    SET prohibited = ["password", "admin", "123456"]
    INPUT number_of_passwords
    IF count is invalid or negative:
        DISPLAY error
        STOP

    valid_count = 0
    invalid_count = 0

    REPEAT for each password:
        INPUT password
        errors = empty list

        IF length(password) < minimum_length:
            ADD length error
        IF no lowercase letter:
            ADD lowercase error
        IF no uppercase letter:
            ADD uppercase error
        IF no digit:
            ADD digit error
        IF no special character:
            ADD special-character error
        IF any character repeats 3 times consecutively:
            ADD repetition error
        IF password contains a prohibited substring (case-insensitive):
            ADD prohibited-substring error

        IF errors is not empty:
            DISPLAY INVALID and every error
            invalid_count = invalid_count + 1
        ELSE:
            DISPLAY VALID
            valid_count = valid_count + 1
    END REPEAT

    DISPLAY total checked, valid_count, invalid_count
END
```

7. Python Source Code

Save the code as **password_policy_validator.py** and run it with Python 3. The code prints validation reasons, not the password text itself.

```
import string

MIN_LENGTH = 8
PROHIBITED_SUBSTRINGS = ["password", "admin", "123456"]


def validate_password(password):
    errors = []

    if len(password) < MIN_LENGTH:
        errors.append(f"Password must contain at least {MIN_LENGTH} characters.")

    if not any(ch.islower() for ch in password):
        errors.append("Add at least one lowercase letter.")

    if not any(ch.isupper() for ch in password):
        errors.append("Add at least one uppercase letter.")

    if not any(ch.isdigit() for ch in password):
        errors.append("Add at least one digit.")

    if not any(ch in string.punctuation for ch in password):
        errors.append("Add at least one special character.")

    # Reject any character repeated three or more times consecutively.
    if any(password[i] == password[i + 1] == password[i + 2]
           for i in range(len(password) - 2)):
        errors.append("Do not repeat the same character 3 or more times in a row.")

    lowered = password.lower()
    for blocked in PROHIBITED_SUBSTRINGS:
        if blocked in lowered:
            errors.append(f"Do not include the prohibited substring: '{blocked}'.")

    return errors


def main():
    print("=== Password Policy Validation Engine ===")
    print(f"Minimum length: {MIN_LENGTH}")
    print("Required: lowercase, uppercase, digit, special character")
    print("Rules: no 3 identical consecutive characters; no blocked substrings")
    print("Enter sample passwords for validation.")

    try:
        count = int(input("How many passwords do you want to check? "))
        if count < 0:
            print("Number of passwords cannot be negative.")
            return
    except ValueError:
        print("Please enter a valid whole number.")
        return

    accepted = 0
    rejected = 0

    for number in range(1, count + 1):
        password = input(f"\nEnter sample password {number}: ")
        errors = validate_password(password)

        # Do not print the password itself; show only the result and reasons.
        if errors:
            rejected += 1
            print("Result: INVALID")
            print("Reasons:")
            for error in errors:
                print(f"- {error}")
        else:
            accepted += 1
            print("Result: VALID")
            print("All password policy rules passed.")

    print("\n=== Validation Summary ===")
    print(f"Passwords checked: {count}")
    print(f"Valid passwords: {accepted}")
    print(f"Invalid passwords: {rejected}")
```

```
if __name__ == "__main__":
    main()
```

8. Test Cases

ID	Sample / Scenario	Expected result
TC1	Abc@1234	Valid: all rules pass
TC2	abc123	Invalid: too short; no uppercase; no special character
TC3	ABCDEFGH1@	Invalid: no lowercase letter
TC4	Abc@aaa123	Invalid: three consecutive 'a' characters
TC5	Admin@1234	Invalid: contains prohibited substring 'admin'
TC6	Abc@123456	Invalid: contains prohibited substring '123456'
TC7	Abc@12xy	Valid: required categories present; no blocked substring or 3-repeat
TC8	Aaa@aaa123	Invalid: repeated 'aaa' sequences
TC9	Count input = -1	Display error and stop
TC10	Password count = 0	Display summary with zero checked

9. Sample Output (Console Screenshot Mock-up)

The transcript below is an illustrative console output, not a screenshot captured from a live terminal. Run the source code to reproduce the result. If your instructor requires real screenshots, capture the running program using your computer's screenshot tool.

```
=== Password Policy Validation Engine ===
Minimum length: 8
Required: lowercase, uppercase, digit, special character
Rules: no 3 identical consecutive characters; no blocked substrings
Enter sample passwords for validation.
How many passwords do you want to check? 4

Enter sample password 1: Abc@1234
Result: VALID
All password policy rules passed.

Enter sample password 2: abc123
Result: INVALID
Reasons:
- Password must contain at least 8 characters.
- Add at least one uppercase letter.
- Add at least one special character.

Enter sample password 3: Abc@aaa123
Result: INVALID
Reasons:
- Do not repeat the same character 3 or more times in a row.

Enter sample password 4: Admin@1234
Result: INVALID
Reasons:
- Do not include the prohibited substring: 'admin'.

=== Validation Summary ===
Passwords checked: 4
Valid passwords: 1
Invalid passwords: 3
```

10. Conclusion

The Password Policy Validation Engine checks sample passwords against multiple configurable rules and explains each failed condition. It demonstrates string handling, loops, conditional statements, lists, and input validation in Python. By reporting all failures rather than stopping at the first one, the program helps users understand which requirements are unmet. The project can be extended with additional rules, a graphical interface, or integration into a properly designed authentication system. It is a learning demonstration and should not be treated as a complete security solution.

How to run the program

- Save the code in a file named password_policy_validator.py.
- Open Command Prompt or a terminal in the file's folder.
- Run: python password_policy_validator.py
- Enter the number of sample passwords and then type each sample when prompted.